An Adaptive FMM-based Inhomogeneous PDE Solver for Complex Geometries

H. Langston[†] D. Zorin L. Greengard

December 8, 2008

Abstract

1 Introduction

Many problems in computational physics and the numerical approaches involve the solutions to elliptic partial diffeerntial equations (PDEs) inside and outside of three-dimensional complex domains subject to Dirichlet boundary conditions where forces are distributed non-uniformly. In particular, consider the Poisson equation,

$$-\Delta u = f, \tag{1}$$

with boundary condition $u|_{\partial\omega} = g$, where f is distributed non-uniformly inside of the three-dimensional shape in figure 1 with domain ω with boundary $\partial\omega$.

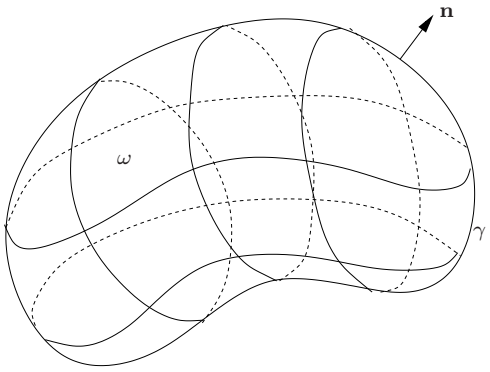


Figure 1: General 3D shape with domain ω and boundary $\partial\omega$.

Related work. n simple rectangular, circular and spherical geometries, well-established fast methods for solving such problems exist. These methods require that the underlying equation can be easily discretized on a regular grid, and they often are based on Cyclic-Reduction or Fast Fourier Transforms (FFT) such as [7]. When geometries become more complex and regular grids are no longer an option, however, these methods are no longer practical.

[†]Courant Institute, New York University, New York 10012. Email: {harper,dzorin,greengard}@cims.nyu.edu
footnotetext

For more complex discretizations, many current solvers rely on adaptive meshes [2], [4] or domain decomposition strategies [10] and multigrid acceleration [18] for solving the Poisson equation [22], [19], [8]. When geometries become more complex, grid-generation techniques become unstructured. Unstructured grid techniques [23] can be very expensive computationally as well as tricky from an algorithmic view-point; generating these grids is often done in parallel [8], [1], which adds additional difficulties. Further, due to multigrid methods, grid-based techniques such as these require maintaining a hierarchy of fine and coarse grids, resulting in multiple regular grid solves at each level. For unstructured meshes, generating these grids at different hierarchies poses additional difficulties.

We mention additional grid methods. The Immersed Boundary Method [28], [29], [30] is a successful grid-based method, but the accuracy is low for our needs, due to the way in which the interface is modeled with delta functions. Another option is regular grids which use stencils to compensate for necessary jumps in potential across boundaries [21] and [9] or [5]. This avoids the necessity for keeping a hierarchy of grids. Unfortunately, for highly nonuniform force distributions, we cannot rely on such grids where distributed forces are expected to be known everywhere. In three dimensions, grid-based methods become increasingly difficult even if forces are uniformly distributed, and these methods can have difficulties for edges and corners in terms of necessary refinement.

Another approach is to incorporate fast direct solvers, which are efficient, scale well in parallel and do not require maintaining the hierarchy of grids. Fast-Multipole Methods (FMM) based solvers have been used for a variety of purposes in two dimensions. In [15], FMM methods are developed for high-order accuracy in unbounded space. The authors of [31] and [32] use adaptive FMM grids and triangulation for solving vortex-based fluid problems. In [27], a fast Poisson solver in two dimensions with complex smooth geometries is generated, and the proposed method will build partially off of this. In [11], innovative precomputation techniques are used for speeding up near-field computation. The basis for FMM and similar fast summation techniques are discussed further in Section **??**. Subsection **??** further discusses the state-of-the-art kernel-independent FMM [36], incorporated into the work here. In Chapter **??** we discussed the advantages of [11] and [12].

The optimal approach is to incorporate grids for corrections for jumps as well as using fast methods where possible. The Embedded Boundary Integral (EBI) approach provides this ability. This allows us to take advantage of integral equations by decoupling volume potential computations from the boundary value problem, which is discretized on a regular grid. This allows for solving a free-space Poisson problem over a complex domain shape, granted the potential goes to zero at the infinity limit. Boundary conditions are forced by adding in a solution to a homogeneous Laplace equation [13]. This decoupling process has been done to second order accuracy in [27] using a uniform mesh; [11] extends this approach by using volume integrals instead of this uniform mesh, allowing for nonuniform force distributions in two dimensions along with higher order accuracy. Additionally, quadrature methods are used there instead of a finite-difference based solver near the boundary.As stated, [27] is a fast solver for complex geometries, which uses an embedding approach in 2D. In [35] the authors apply this approach to Stokes equations for slow-moving smooth interfaces in two dimensions.

In general , an integral-equation based method for the solution of elliptic PDE in three dimensions would add value in several areas, including:

- Ability to handle highly non-uniform force distributions. Using adaptive methods based on integral equations without grids, current methods, which rely on uniform force distributions for solving, can be improved.
- Ability to handle complex geometries. Current methods support fast methods for arbitrarily smooth geometries, which can be incorporated.
- High accuracy. Using methods based in potential theory allows for direct evaluation of the solution at a non-uniform distribution of target points, and fast methods incorporate high accuracy quickly.
- In the use of the kernel-independent Fast Multipole Method for the Poisson equation, an added benefit is the straightforward extension from the Poisson equation to other elliptic PDEs.

Our approach is based on the EBI approach from the work of [35] and [24]. For distributed forces available everywhere, [37] discretizes the regular domain for an FFT-solver. For a non-uniform force distribution, this approach is inadequate. We use the volume integral approach of [12] and [11] along with the kernel-independent Fast Multipole Method [36],[38]. For smooth boundaries, Nyström methods are available along with nearly singular integration for evaluation of potentials near the boundary [37], [6].

In the use of the kernel-independent Fast Multipole Method for the Poisson equation, an added benefit is the straightforward extension from the Poisson equation to other elliptic PDEs. In particular, we will look at the following three equations for a variety of complex geometries:

- The Poisson Equation: $\Delta u = f$
- The Modified Poisson Equation: $\alpha u - \Delta u = f$
- The Stokes Equations: $\nabla p - \Delta \mathbf{u} = \mathbf{f}, \nabla \cdot \mathbf{u} = 0$

These three example problems allow the opportunity to study equations in the kernel representation sense with single degrees of freedom (Laplace), multiple degrees of freedom (Stokes), scale invariance (Laplace and Stokes) and scale variance (Modified Laplace). Hence, these three kernels are representative of all other possible kernels, best exemplifying the strength of the method and approach. The extension to more complex elliptic PDE (i.e., Modified Stokes equations) is a straightforward extension of the methods developed for the equations above.

The paper is organized as follows. We will begin by discussing the embedded boundary integral (EBI) approach in the next section. Following this, review our fast volume solver based on integral-equation approaches and accelerated with FMM from [20]. We then review the boundary integral formulation and quadrature methods of [37]. sectionEmbedded Boundary Integral Formulation for Inhomogeneous PDEs

For solving complicated problems such as those in figure 1, we discuss the Embedded Boundary Integral (EBI) approach as in [35]. For an inhomogeneous interior Dirichlet PDE of the form

$$\mathcal{L}(u)(\mathbf{x}) = f(\mathbf{x}) \ \text{ in } \omega, \ u = g \ \text{ on } \partial\omega \tag{2}$$

do the following:

1. Embed ω in a regular domain, Ω with domain $\partial\Omega$ and solve:

$$\mathcal{L}(u_1)(\mathbf{x}) = f(\mathbf{x}) \ \text{ in } \Omega, \ u_1 \to 0 \ \text{ as } \mathbf{x} \to \infty \tag{3}$$

where the forces f are given at possible irregular locations, depending on the input.

2. Solve a boundary integral problem on $\partial\omega$ where the border data, g is modified using the solution of u_1 derived from step 1. Given that we solve for u_1 everywhere using the FMM method, we solve for u_1 on $\partial\omega$ at the same time we solve for u_1 everywhere else in Ω.

3. Solve the homogeneous Laplace equation:

$$\mathcal{L}(u_2)(\mathbf{x}) = 0, \ u_2 = g - u_1 \ \text{ on } \partial\omega \tag{4}$$

4. Add u_1 and u_2 to obtain the full solution.

In previous work, the prism, in which ω is embedded was discretized regularly such that $h_x = h_y = h_z$ [25, 24, 26, 27]. As suggested in [24], $\partial\Omega$ is chosen to be far enough removed from $\partial\omega$. For our approach, u_2 will not have to be evaluated on $\partial\Omega$, so it need not have a regular discretization, so we can choose $\partial\Omega$ to be a bounding box of arbitrary size as long as it encloses all of ω. As a graphic example, figure 2 takes the shape in figure 1 and embeds it inside of a rectangular prism, which has been regularly discretized (again, this is not necessary).

In the next section, we discuss how we can solve 3 everywhere in Ω using FMM-based volume methods.

2 Inhomogeneous and Homogeneous PDE Solvers

Based on the above, we need to solver an inhomogeneous PDE in a simple geometry and a homogeneous PDE in a complex geometry. For the first, we use our previously-developed fast kernel-independent fast volume solver [20], and for the second, we use the boundary integral solver and nearly-singular integration methods of [35]. We begin by reviewing the fast volume solver.

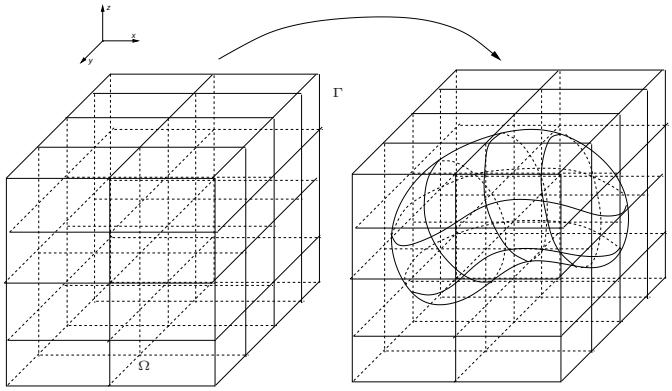


Figure 2: The original domain is embedded in a uniformly-discretized domain.

2.1 Fast Volume Solver

In free space, linear constant-coefficient PDE solutions can be expressed directly in an integral form; that is, for a given location, the solution of the equation can be evaluated as a convolution of the volume force with a specific Green's function. By taking advantage of smoothness of the kernel functions, FMM algorithms [3, 12, 16] are able to significantly decrease complexity costs. In particular, the kernel-independent FMM [36, 38] makes it possible to efficiently solve any free-space PDE for which a smooth kernel evaluation function is provided.

Given an elliptic PDE of the form

$$\mathcal{L}(u)(\mathbf{x}) = f(\mathbf{x}), \tag{5}$$

we can solve equation (5) in free-space by evaluating the analytic solution,

$$u(\mathbf{x}) = \int_{\mathcal{R}^d} K(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{y}, \tag{6}$$

We use an adaptive piecewise polynomial force approximation, discussed in detail in section **??** As the input for our algorithm, we assume a smooth force is given as discrete point sources, which can in turn be approximated locally as polynomial coefficients. This approximation can be constructed from other types of force representations as long as the forces can be evaluated at grid enough locations with sufficient accuracy ([20]).

Given a force distribution f at N_{src} source locations, we wish to compute the induced potentials u_j at N_{trg} target locations, $\mathbf{x}_j$:

$$u_j = u(\mathbf{x}_j) = \int_{\mathcal{R}^d} K(\mathbf{x}_j, \mathbf{y}) g(\mathbf{y}) d\mathbf{y} \approx \sum_{i=1}^{N_{src}} K(\mathbf{x}_j, \mathbf{y}_i) g(\mathbf{y}_i) w_i \ , \ j = 1, \ldots, N_{trg} \, , \tag{7}$$

where $K(\mathbf{x}, \mathbf{y}) = -\log |\{\mathbf{x} - \mathbf{y}\}| / 2\pi$ and w_i is a quadrature weight associated with source location $\mathbf{y}_i$. For $N_{src} \approx N_{trg} \approx N$, the FMM decreases the computational cost from $O(N^2)$ to $O(N)$ for a fixed user-prescribed level of accuracy by first partitioning space in a hierarchical sense for categorizing the relative proximity of points. In order to achieve this goal, we assume that the points can be contained within a finite regular domain, D (here, we assume that D is a rectangle in 2D and a box in 3D). To partition our data set inside of D, we use a hierarchical space partitioning algorithm, represented as a quadtree in 2D and an octree in 3D. The method is designed to simply subdivide D and all of descendants into 2^d equivalently-sized regions until each box B, represented as a leaf in the tree data structure T, contains at most s points. For a cubic region, D of size $[-1/2, 1/2]^3$, a resulting leaf box B at level ℓ in T will have width 2^ℓ. We will continue to assume that D is a box of this size for our discussion.

We further designate a number of lists for each box B in T. These lists are mainly designed to assist in organization of relative data. For the purposes of the Fast Multipole (FMM) algorithm, these lists will be used for locating data

necessary for evaluating potentials in the solution phase. For each box B, we define a *near field*, $\mathcal{N}^B$, and a *far field* of B, $\mathcal{F}^B$. $\mathcal{N}^B$ is the list of all boxes in D enclosed inside of a box centered at B with width $3 \cdot 2^\ell$, and $\mathcal{F}^B$ is defined as $D/\mathcal{N}^B$. For the FMM, we also define the *neighbor list* L_N^B (boxes sharing a vertex with B such that $L_N^B \subseteq \mathcal{N}^B$), and the *interaction list* L_I^B (children of B's parent's neighbors not in L_N^B such that $L_I^B \subseteq \mathcal{F}^B$. For a uniformly-refined tree (all leaves at the same height), these are all of our lists, and an example 2D domain and quadtree structure can be seen in Figure 3.

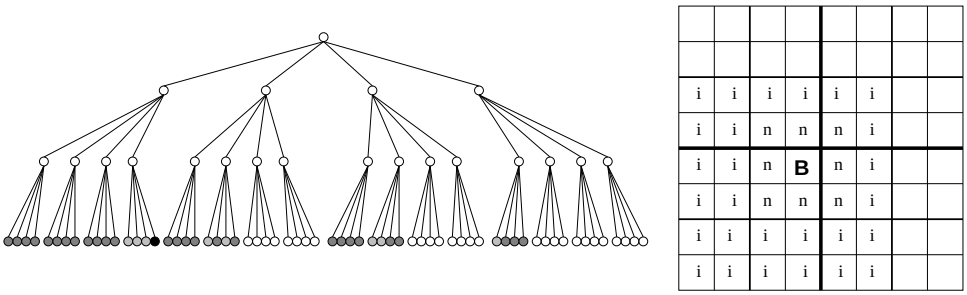


Figure 3: On the *right* is shown a domain which has been fully subdivided three times, resulting in a quadtree on the *left*. The root of the tree represents the entire domain at level 0. The non-leaf nodes at level 1 of the quadtree represent the four boxes on the right enclosed by the thickest black lines. The right image shows the neighbor list L_N^B for the box B (boxes marked with *n* on the grid, light gray in the tree), and L_I^B (boxes marked with *i* on the grid, dark gray in the tree). Here, $\mathcal{N}^B = L_N^B$ and $\mathcal{F}^B = D \cup L_I^B$.

For a non-homogeneous distribution of points, uniformly-refined trees are often not possible; as a result, several additional lists must be used to organize our data. Our definitions and notation follow [14, 16, 17].

For a leaf box B, let the *U-list*, L_U^B, be the set of other leaves adjacent (at arbitrary levels) to B, including itself; L_U^B coincides with the neighbor list L_N^B for the uniformly refined case. The *W-list*, L_W^B, is defined to be the set of descendants of B's neighbors, not adjacent to B, but whose parents are adjacent to B. This list contains boxes at finer levels than B for which B is in their far range, but which are in the near range of the parent of B. For any $W \in L_W^B$, W is at a finer level than B and $W \in \mathcal{N}^B$ (conversely, $B \in \mathcal{F}^B$). For leaf and non-leaf boxes B, the *V-list*, L_V^B, is the set of B's parent's neighbor's children, not-adjacent to B. $L_V^B = L_I^B$ for uniformly refined trees. Finally, the *X-list*, L_X^B, is the set of boxes, A such that $B \in L_W^A$.

We note the following: $B \in L_U^A$ if and only if $A \in L_U^B$, $V \in L_V^A$ if and only if $A \in L_V^B$, $B \in L_W^A$ if and only if $A \in L_X^B$. A fragment of an adaptively refined level-restricted tree these lists is shown in Figure 4.

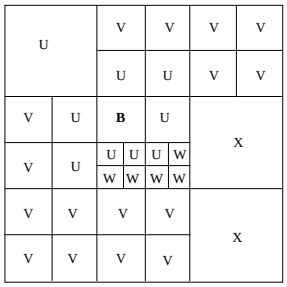


Figure 4: A 2d quadtree resulting from a non-uniform source distribution. For box B, correspondingly marked boxes represent the boxes in the U, V, W, and X lists of B.

Additionally, we note that in the evaluation stage of our implementation of the FMM algorithm, we require that any two adjacent leaf boxes in T are within one level of each other in the tree. This is a common *tree-level restriction* (***** citations *****). Many fast approaches exist to convert arbitrary octrees to ones satisfying this constraint [33]; however, we currently use a straightforward sequential algorithm as discussed in [20].

The FMM says that for some $\mathbf{x} \in B$ for a box B, (6) can be approximated as:

$$u(\mathbf{x}) = \int_{\mathcal{N}^B} K(\mathbf{x},\mathbf{y}) f(\mathbf{y}) d\mathbf{y} + \int_{\mathcal{F}^B} K(\mathbf{x},\mathbf{y}) f(\mathbf{y}) d\mathbf{y} \tag{8}$$

The first near-field component uses pre-computed weights and polynomial approximations to f to compute the contribution to u at $\mathbf{x}$, while the far-field component uses equivalent density reprsentations, computed at the surface of V to compute the influence from B's interaction list. These equivalent representations are computed using V's polynomial approximation to its support of f. All other interactions are accounted for from B's ancestors. The details for the precomputed weights are avilable in [20] while the equivalent densities are a component of the kernel-independent FMM [36, 38], also discussed in detail for the purposes of volume integrals in [20]. The key point here is that we require an accurate polynomial representation at every leaf box B of f^B, B's support of the force f. We will refer to this polynomial as γ^B.

To motivate the construction of γ^B, we begin with an example in which g^B is given on grid of size $k \times k \times k$ (hereafter referred to as a k^3 grid). A kth order approximation, γ^B could then be constructed as

$$g^B(\mathbf{x}) \approx \sum_{j=1}^{N_k} \gamma_j^B \beta_j(\mathbf{x} - \mathbf{c}_B), \tag{9}$$

where $N_k = k(k+1)(k+2)/6$, and β_j is the jth basis polynomial in our representation (for $k = 4, 6$, we use monomials and require k^3 equispaced points in practice and for $k > 6$, we use Chebyshev polynomials and Chebyshev points; motivation for this requirement is discussed in[20]). Clearly this system is ovedetermined [34], so we can solve (9) for γ^B using the SVD.

In general, the above approach does not work for obvious reasons. Since forces may not always be represented on a grid, so we may not have enough points to accurately and stably compute γ^B especially since we will require our tree to be balanced; that is, assuming T has been subdivided to have at most s points per leaf box B, subsequent balancing for tree-level restriction may result in some leaves with very few or no points. To compensate for such leaves, we utilize the data structures available to us through the various lists to build γ^B at sufficiently high order. Further, even for forces on a grid, we may have a situation in which a leaf node, B, intersects the curve and only a fraction of its grid points are contained within the computational domain. For example, consider difure 5, where several leaf nodes lack enough grid points to accuractely compute the polynomial approximation to the degree requested.

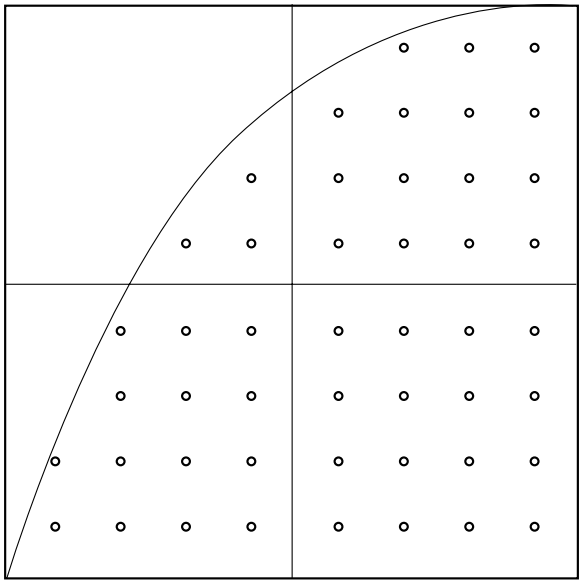


Figure 5: Caption

We construct a *coefficient-to-coefficient mapping* approach. Let B be a box of half-width r (a single side is of length $2r$) such that B has fewer than N_{min} points (in practice, we set $N_{min} >= k^3$ for k being the order of approximation). We then attempt to construct a region, $\tilde{B}$ such that $B \subset \tilde{B}$, $\tilde{B}$ contains at least N_{min} points (typically set to actually be $2N_{min}$), and $\tilde{B}$ is a spherically symmetric region (about B's center $\mathbf{c}_B$) of radius $c \times r$ for some constant c.

The reason we choose to search in a spherically symmetric fashion is that since the force distribution g in D should be smooth, we want to avoid any *dropoff* on a face of B (***** I am trying to explain that I think going up to B's parent is not a spherically symmetric approach and could misrepresent the edge of the force along a face or edge of B - Maybe some examples will need to be generated for comparing using $\mathcal{N}^B$ versus B's parent *****). We also note that B's center $\mathbf{c}_B$ will also be the center of $\tilde{B}$, simplifying our fomulations.

Fortunately, our lists L_U^B, L_W^B, L_X^B, and L_V^B provide most of the information needed. If there are not enough points in these boxes, we can investigate the boxes in L_V^P for B's parent, P, since it is clear that L_V^P is disjoint from the union of L_U^B, L_W^B, L_X^B, and L_V^B. We begin with all leaf boxes in L_U^B, L_W^B, L_X^B simply because for memory purposes, only leaf boxes store the locations and values of force densities, so for traversing boxes in L_V^A for some box (leaf or non-leaf) A, we must build a list of points contained within the appropriate domain.

Assuming that we can construct $\tilde{B}$ which contains the necessary number of points, let $g^{\tilde{B}}$ be the respective force distribution, represented at discrete points. We then build an approximation to $g^{\tilde{B}}$ as

$$g^{\tilde{B}}(\mathbf{x}) \approx \sum_{j=1}^{N_k} \gamma_j^{\tilde{B}} \beta_j(\mathbf{x} - \mathbf{c}_B). \tag{10}$$

Since $B \subset \tilde{B}$, $\gamma^{\tilde{B}}$ can be evaluated inside of B. That is, let $g^{B,g}$ be the restriction of $g^{\tilde{B}}$ onto a k^3 grid on B, located at grid points $\mathbf{x}^{B,g}$ such that we compute

$$g^B(\mathbf{x}) = \sum_{j=1}^{N_k} \gamma_j^{\tilde{B}} \beta_j(\mathbf{x} - \mathbf{c}_B) \quad \text{for all } \mathbf{x} \in \mathbf{x}^{B,g}. \tag{11}$$

We note that since $\tilde{B}$ was constructed to be symmetric about B's center $\mathbf{c}_B$, we still evaluate $\gamma^{\tilde{B}}$ about $\mathbf{c}_B$. Given a representation of g restricted to B in the form of $g^{B,g}$, we now use (9) to construct γ^B, and given the size of $g^{B,g}$, we are guaranteed that the system is sufficiently over-determined.

We outline the steps algorithmically in the following pseudocode. We assume that T has already been built as well as all lists for a box B.

We note that in practice, we search B's parent's interaction list, and if we are unable to collect enough points after traveling this far in the tree, we reduce the order of our polynomial as necessary. If we cannot even locate a single point, B's coefficients are all zero, and B is ignored in the computation stage.

For a uniformly-refined tree, figure 6 illustrates the coefficient construction process.

3 Embedded Boundary Integral Tests and Results

We begin by testing the accuracy of the boundary integral solver and the fast volume solver separately for convergence rates. For both tests, we begin by building uniformly refined octrees within our domain such that we can equally space points within leaves of the octree for all leaf boxes which lie in the interior or overlap the geometry. For example, in figure 7, we show a uniformly-refined tree and the domain of computation for the free-space volume solver.

In order to study the convergence rates and appropriate discretizations for our domain and boundary, we begin by looking at the fast volume solver and boundary integral solvers separately. For both tests, we look at the complex domain ω in figure 8a) which has been embedded in a box of size $[-1, 1]^3$; the domain is refined in a non-adaptive regular manner in order to understand the effects of regular grid-spacing.

3.1 Convergence Test for Boundary Integral Solver

We begin by studying the convergence rates for the boundary integral solver by investigating a case in which the body forces are set to zero for the Poisson equation:

Algorithm 1 Build Coefficients for a box B with half-width r

STEP 1 - LOCATE INTERPOLATION POINTS
Let $N = 0$ and iterate through $\mathcal{N}^B$ and L_I^B
for each list $L = L_U^B, L_W^B, L_X^B, L_V^B$ **do**
  while $N < N_{min}$ **do**
    for $\mathbf{x} \in A$ with force density $g^B(\mathbf{x})$ for each box $A \in L$ **do**
      store $\mathbf{x}$ in $P[\lfloor \mathbf{x} - \mathbf{c}_B \rfloor]$
      N = N+1
    end for
  end while
end for
if $N < N_{min}$ **then**
  Let $P = Parent(B)$
  while $P \neq Root(T)$ AND $N < N_{min}$ **do**
    for $\mathbf{x} \in A$ with force density $g^B(\mathbf{x})$ for each box $A \in L_V^P$ **do**
      store $\mathbf{x}$ in $P[\lfloor \mathbf{x} - \mathbf{c}_B \rfloor]$
      N = N+1
    end for
    $P = Parent(P)$
  end while
end if
Let $\tilde{B}$ be the region enclosing all point forces $g^{\tilde{B}}(\mathbf{x})$ in $P[0]$ to $P[c]$ where $\tilde{B}$ has radius $c \times r$
STEP 2 - COMPUTE APPROXIMATION FOR B
Let $c = length(P)$
Compute $\gamma^{\tilde{B}}$ from $g^{\tilde{B}}$ using (10)
Evaluate g^B at $\mathbf{x}^{B,g}$ from $\gamma^{\tilde{B}}$ using (11)
Compute γ^B from g^B using (9)

$$-\Delta u(\mathbf{x}) = 0, \mathbf{x} \in \omega$$
$$u|_\gamma = 1 + x * y * z$$

This is a repeat of a similar test from [36], and tests for the individual portions of the singular and nearly singular portions of the solver can be seen there. Here, we are mainly interested in the total error seen in the boundary integral solver for comparison with the fast volume solver.

For the interior grid of points in ω we choose a grid discretization h which corresponds to the distance between all points in an equally spaced grid on a leaf box, B in our tree T. That is, for a box B with k^3 equispaced gridpoints, $\mathbf{x}^{B,g}$, if all leaves in T are at depth d, then the distance beteween adjacent points $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ is $h = 2^{-(d-1)}/k$. This Cartesian grid of points is generated for all leaf boxes which lie inside of ω or overlap γ; an example slice of these gridpoints can be seen in figure 8b). The spacing for the discretization of γ is also set to h to ensure, as in [36] that some gridpoints are within a distance less than h from γ, enforcing that some potential values will be evaluated using nearly singular integration. In our tests we set $\epsilon_{fmm} = 10^-8$ for all tests and $k = 4$ such that $h = 2^{-(d_{max}+1)}$.

h	d_{max}	N_{pts}^{ω}	N_{pts}^{γ}	ϵ_2^u	Rate	ϵ_∞^u	Rate
0.1250	2	5120	1080	$7.0\ 10^{-3}$		$2.0\ 10^{-2}$	
0.0625	3	10240	4144	$4.4\ 10^{-4}$	4.0	$3.4\ 10^{-3}$	2.6
0.03125	4	53760	15912	$5.3\ 10^{-5}$	3.1	$7.0\ 10^{-4}$	2.3
0.015625	5	353280	62340	$3.9\ 10^{-6}$	3.8	$8.2\ 10^{-5}$	3.1
0.0078125	6	2535424	249468				

Table 1:

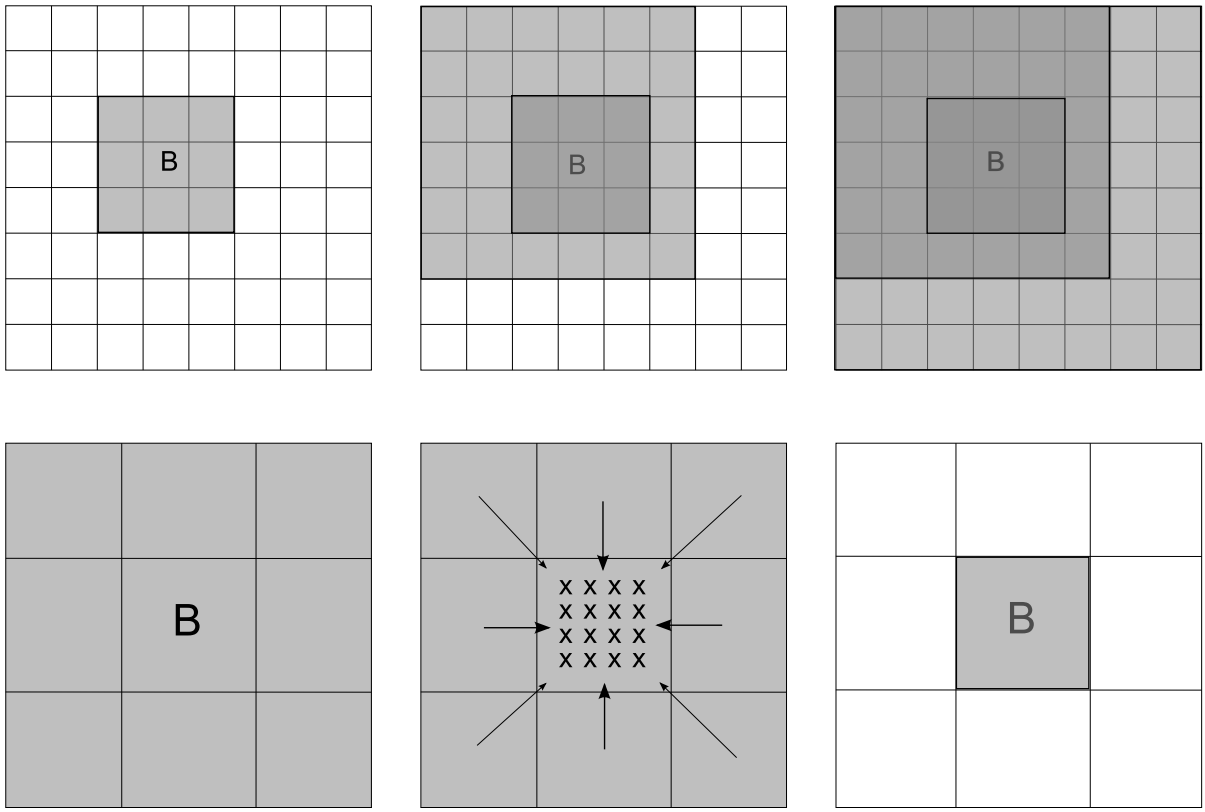


Figure 6: *Top Row* : for a box B, we assume B does not have a sufficient number of points, so we look at the boxes in its near-field (marked on the *left*). If there are still not enough points, we search through B's interaction list (indicated in the *middle*). While there are still not enough points to build an interpolant, we recursively search B's ancestors' interaction lists (indicated on the *right*). *Bottom Row*: assuming the region marked contains a sufficient number of points (in this case, this region is $\mathcal{N}^B$), let this larger region be $\tilde{B}$. Build an interpolant to the forces in $\tilde{B}$ on the *left*. In the *middle*, we evaluate this polynomial approximation at grid points $\mathbf{x}^{B,g}$ in B. On the *right*, we use the evaluated grid-located forces to compute B's local approximation to the force distribution. ***** When I print this, the image looks incorrect - I am checking it (may be layers in inkscape) *****

3.2 Convergence Tests for Free-Space Fast Volume Solver

For the fast volume solver convergence tests, we look solely at the domain ω absent the geometry γ. That is, let $\tilde{T}$ be the portion of the tree which lies inside of or touches ω For the purposes of this test, we assume that we are given the values of the force distribution, f at all grid points $\mathbf{x}^{B,g}$ for all leaves, B which are inside of or overlap ω. For illustration purposes, we return to figure 7a) and b). In these images, we see a 2d example of a circle embeddded in a regularly subdivided quadtree. Each of the smallest boxes represents a k^3 grid for some value k, on which we are given discrete values for f and wish to evaluate the discrete volume potential, u. Figure b) outlines the region of evaluation.

For convergence studies, we will *fix* a non-adaptive octree T for some depth d, and for each leaf box $B \in \tilde{T}$, we build a kth order polynomial approximation, γ^{B_k} of size $N_k = k(k+1)(k+2)/6$ for some value k, representing the size of the k^3 grid. We then use these polynomials to evaluate the volume potential $u^k(\mathbf{x})$, assuming $u^k \to 0$ as $\mathbf{x} \to \infty$ as described above.

In order to studey the errors of the solutions, for a fixed-level treet T with evaluation region, $\tilde{T}$, we compute $u^{k_{max}}$ for $k_{max} = 12$ and compare this to u^k for $k = 4, 6, 8$.

For these tests, we use the following complex test force for the Poisson equation. The purpose of this force is to ensure that we can approximate f well with the approximation of order k_{max} but within the requested order for $k = 4, 6, 8$

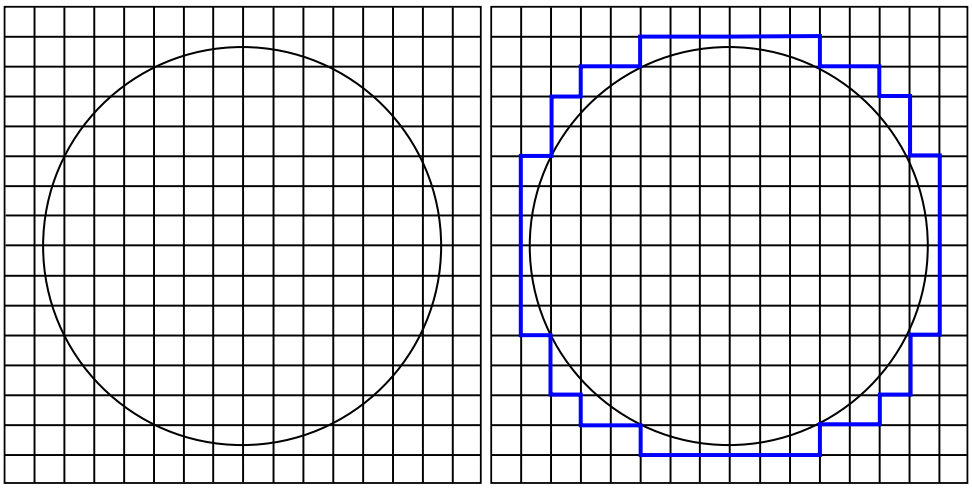


Figure 7: Caption

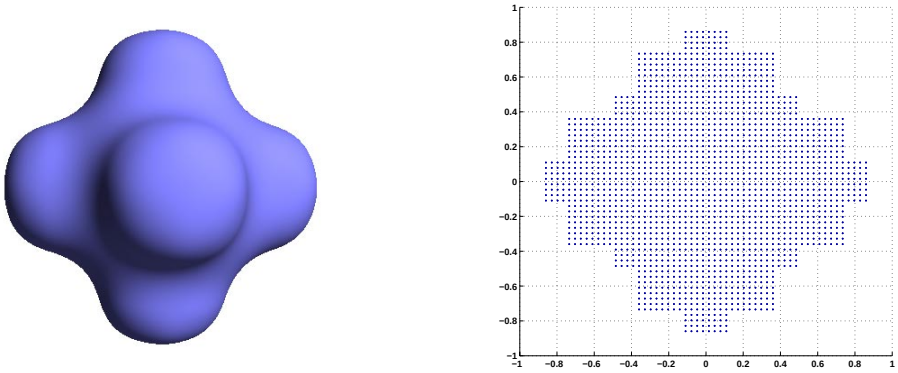


Figure 8: Caption

$$-\Delta u(\mathbf{x}) = g(\mathbf{x}), \tag{12}$$

$$g(\mathbf{x}) = e^{-\mathbf{x}^T \cdot \mathbf{x}\sqrt{2}} \sin 3\pi \, ||\mathbf{x}||^2 + (1/396)(\mathbf{x} \cdot \mathbf{e})^{12}$$

We further note that for $k = 8$, we use Chebyshev grid points to solve for the potential; however, we then interpolate these values from the Chebyshev grid $\mathbf{x}^{B,g}$ on each leaf to an equispaced grid on B, where h indicates the distance between points.

3.3 Convergence Tests for Polynomial Coefficient Mapping

In the previous tests, we assumed that we were give the force at discrete points for all leaf boxes $B \in \tilde{T}$, include points which lay outside of ω where B overlapped the boundary γ. Now, we assume we are only given a complete grid for B which lie completely inside of the interior of ω. For boxes which overlap the boundary, we are only given those points which lay in the interior. For example, again consider figure 5. These overlapping boxes are the results of locating where B intersects γ as seen in figure **??**b).

h	$\tilde{T}_d$	$N_{pts}^{\tilde{T}}$	$\epsilon_2^{u^k}$	Rate	$\epsilon_\infty^{u^k}$	Rate
$k = 4$, Fourth-Order Force Approximation						
0.0625	3	10240	$1.4\ 10^{-3}$		$7.4\ 10^{-3}$	
0.03125	4	53760	$8.9\ 10^{-4}$		$3.3\ 10^{-3}$	
0.015625	5	353280	$4.1\ 10^{-4}$		$1.2\ 10^{-3}$	
0.0078125	6	2535424	10		10	
$k = 6$, Sixth-Order Force Approximation						
0.08333	2	6912	$1.4\ 10^{-3}$		$8.3\ 10^{-3}$	
0.04167	3	34560	$1.1\ 10^{-4}$		$6.3\ 10^{-4}$	
0.02083	4	181440	$8.5\ 10^{-5}$		$3.8\ 10^{-4}$	
0.010417	5	1192320	$3.6\ 10^{-5}$		$1.2\ 10^{-4}$	
0.0052083	6	8557056				
$k = 8$, Eighth-Order Force Approximation						
0.0625	2	16384	$6.4\ 10^{-3}$		$7.8\ 10^{-3}$	
0.03125	3	81920	$3.7\ 10^{-4}$		$9.0\ 10^{-4}$	
0.015625	4	430080	$2.7\ 10^{-5}$		$8.1\ 10^{-5}$	
0.0078125	5	2826240	10		10	
0.00390625	6					

Table 2:

To test the accuracy of our interpolation and coefficient-mapping procedure from section **??**, we again fix a tree for specific depths d_{max}. For boxes, $\tilde{B}$ which overlap the boundary, we compute interpolated polynomial approximations, $\tilde{\gamma}^B$ and then evaluate the following,

$$f(\mathbf{x}) = \sum_{j=1}^{N_k} \beta_j(\mathbf{x} - \mathbf{c}_B)\tilde{\gamma}_j^B,$$

for all $bx \in \partial\omega$ in order to test the accuracy of the interpolated polynomials at the boundary. These points lie outside of all interior points and are not on our grids, $bx^{B,g}$, providing good locations for testing local polynomials. We compute the relative pointwise error at all such points as well as the relative maximum error. Additionally, we recompute the free-space volume solution, u^k as in the previous example and compare this to $u^{k_{max}}$, where for $k_{max} = 12$, we are given f at all grid points, including overlapping ones (no interpolation needed). This allows us to compare the accuracy of u^k when interpolation is needed versus when it is not.

4 Conclusions

References

[1] M. F. Adams and J. Demmel. Parallel multigrid solver algorithms and implementations for 3D unstructured finite element problem. In *ACM/IEEE Proceedings of SC99: High Performance Networking and Computing*, Portland, Oregon, November 1999.

[2] M.J. Aftosmis, M.J. Berger, and J.E. Melton. Adaptive Cartesian mesh generation. In J.F. Thompson, editor, *The Handbook of Grid Generation*, pages 22–1–22–26. CRC Press, 1998.

[3] R. Beatson and L. Greengard. A short course on fast multipole methods. In M. Ainsworth et al., editors, *Wavelets, multilevel methods and elliptic PDEs*, pages 1–37, Walton Street, Oxford OX2 6DP, UK, 1997. Oxford University Press.

[4] M.J. Berger, M. Aftosmis, and J. Melton. Accuracy, adaptive methods and complex geometry. In *1st AFOSR Conference on Dynamic Motion CFD*, 1996.

[5] M.J. Berger and P. Colella. Local adaptive mesh refinement for shock hydrodynamics. *J. Comput. Phys.*, 82:64–84, 1989.

[6] Oscar P. Bruno and Leonid A. Kunyansky. A fast, high-order algorithm for the solution of surface scattering problems: Basic implementation, tests, and applications. *J. Comput. Phys.*, 169:80–110, 2001.

h	$\tilde{T}_d$	$N_{pts}^{\tilde{T}}$	ϵ_2^f	Rate	ϵ_∞^f	Rate	$\epsilon_\infty^{u^k}$	Rate
$k = 4$, Fourth-Order Force Approximation								
0.0625	3	10240	$1.1\ 10^{-2}$		$1.4\ 10^{-3}$		$3.1\ 10^{-1}$	
0.03125	4	53760	$1.0\ 10^{-3}$		$1.8\ 10^{-4}$		$4.1\ 10^{-2}$	
0.015625	5	353280	$1.0\ 10^{-4}$		$2.2\ 10^{-5}$		$3.8\ 10^{-3}$	
0.0078125	6	2535424	10		10			
$k = 6$, Sixth-Order Force Approximation								
0.08333	2	6912	$1.1\ 10^{-2}$		$1.4\ 10^{-3}$		$1.0\ 10^{-1}$	
0.04167	3	34560	$4.2\ 10^{-4}$		$9.2\ 10^{-5}$		$6.7\ 10^{-3}$	
0.02083	4	181440	$1.3\ 10^{-5}$		$3.6\ 10^{-6}$		$7.4\ 10^{-5}$	
0.010417	5	1192320	$3.6\ 10^{-7}$		$1.0\ 10^{-7}$		$2.6\ 10^{-5}$	
0.0052083	6	8557056						
$k = 8$, Eighth-Order Force Approximation								
0.0625	2	16384	$2.2\ 10^{-3}$		$3.0\ 10^{-4}$		$6.0\ 10^{-2}$	
0.03125	3	81920	$1.3\ 10^{-5}$		$2.3\ 10^{-6}$		$1.1\ 10^{-3}$	
0.015625	4	430080	$1.9\ 10^{-7}$		$8.6\ 10^{-7}$		$9.6\ 10^{-5}$	
0.0078125	5	2826240	10		10			
0.00390625	6							

Table 3:

[7] B.L. Buzbee, G.H. Golub, and C.W. Nielson. On direct methods for solving Poisson's equation. *SIAM J. Numer. Anal.*, 7:627–656, 1970.

[8] T. Chan and B. Smith. Domain decomposition and multigrid algorithms for elliptic problems on unstructured meshes. *Electron. Trans. Numer. Anal.*, 1994:171–182, 1994.

[9] L.T. Cheng, R.P. Fedkiw, F. Gibou, and M. Kang. A second order accurate symmetric discretization of the Poisson equation on irregular domains. *J. Comput. Phys.*, 176:205–227, 2001.

[10] G. Chesshire and W. Henshaw. Composite overlapping meshes for the solution of partial differential equations. *J. Comput. Phys.*, 90:1–64, 1991.

[11] F. Ethridge. *Fast Algorithms for Volume Integrals in Potential Theory*. Ph.d. thesis, New York University, New York, NY, 2000.

[12] F. Ethridge and L. Greengard. A new fast-multipole accelerated Poisson solver in two dimensions. *SIAM J. Sci. Comput.*, 23(3):741–760, May 2001.

[13] A. Greenbaum, L. Greengard, and G.B. McFadden. Laplace's equation and the dirichlet-neumann map in multiply-connected domains. *J. Comput. Phys.*, 105(2):267–278, 1993.

[14] L. Greengard. Fast algorithms for classical physics. *Science*, 265(5174):909–914, August 1994.

[15] L. Greengard and Lee J, Y. A direct adaptive Poisson solver of arbitrary order accuracy. *J. Comput. Phys.*, 125:415–424, 1996.

[16] L. Greengard and V. Rokhlin. A fast algorithm for particle simulations. *J. Comput. Phys.*, 73:325–348, August 1987. This paper is credited as the origin of the fast multipole method, with an $O(N)$ algorithm. It was reprinted in the same journal, vol. 135, pp. 280–292, August 1997.

[17] L. Greengard and V. Rokhlin. The rapid evaluation of potential fields in three dimensions. In C. Anderson and C. Greengard, editors, *Vortex Methods*, Lecture Notes in Mathematics. Springer Verlag, N.Y., 1988.

[18] W. Hackbusch and U. Trottenberg. *Multigrid Methods, Lecture Notes in Mathematics Volume 960*. Springer-Verlag, 1st edition, 1982.

[19] H. Johansen and P. Colella. A Cartesian grid embedded boundary method for Poisson's equation on irregular domains. *J. Comput. Phys.*, 147:60–85, 1998.

[20] H. Langston, L. Greengard, and D. Zorin. A free-space adaptive fmm-based pde solver in three dimensions. 2008.

[21] Xu-Dong Liu, Ronald P. Fedkiw, and Mypaper.texungjoo Kang. A boundary condition capturing method for Poisson's equation on irregular domains. *J. Comput. Phys.*, 160:151–178, 2000.

[22] D. Martin and K. Cartwright. Solving Poisson's equations using adaptive mesh refinement. Technical Report M96/66, University of California, Berkeley Electronic Research Laboratory, 1996.

[23] D.J. Mavripilis. Unstructured grid techniques. *Annu. Rev. Fluid Mech.*, 29:473–514, 1997.

[24] A. Mayo. The fast solution of Poisson's and the biharmonic equations on irregular regions. *SIAM J. Numer. Anal.*, 21(2):285–299, April 1984.

[25] A. Mayo. Fast high order accurate solution of Laplace's equation on irregular regions. *SIAM J. Sci. Comput.*, 6(1):144–157, January 1985.

[26] A. Mayo and A. Greenbaum. Fast parallel iterative solution of Poisson's and the biharmonic equations on irregular regions. *SIAM J. Sci. Comput.*, 13(1):101–118, January 1992.

[27] A. McKenney, L. Greengard, and A. Mayo. A fast Poisson solver for complex geometries. *J. Comput. Phys.*, 118:348–355, 1995.

[28] Charles S. Peskin. Numerical analysis of blood flow in the heart. *J. Comput. Phys.*, pages 200–252, 1977.

[29] Charles S. Peskin and David M. McQueen. A three-dimensional computational method for blood flow in the heart. I: Immersed elastic fibers in a viscous incompressible fluid. *J. Comput. Phys.*, 81:372–405, 1989.

[30] Charles S. Peskin and B.F. Prinz. Improved volume conservation in the computation of flows with immersed elastic boundaries. *J. Comput. Phys.*, 105:113–132, 1993.

[31] G. Russo, J.A. Strain, and V. Vetoio. Fast triangulated vortex methods for the 2-D euler equations. *J. Comput. Phys.*, 111:291–323, 1994.

[32] J. Strain. Fast adaptive 2D vortex methods. *J. Comput. Phys.*, 132:108–122, 1997.

[33] H. Sundar, R. Sampath, and G. Biros. Bottom-up construction and 2:1 balance refinement of linear octrees in parallel. *Submitted: SIAM J. Sci. Comput.*, 2007.

[34] L.N. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM, 1997.

[35] L. Ying, G. Biros, and D. Zorin. A fast solver for the Stokes equations with distributed forces in complex geometries. *J. Comput. Phys.*, 194(1):317–348, 2004.

[36] L. Ying, G. Biros, and D. Zorin. A kernel-independent adaptive fast multipole method in two and three dimensions. *J. Comput. Phys.*, 196(2):596–626, 2004.

[37] L. Ying, G. Biros, and D. Zorin. A high-order boundary integral equation solver for elliptic pdes in smooth domains. *To Appear: J. Comput. Phys.*, 2006.

[38] L. Ying, G. Biros, D. Zorin, and M.H. Langston. A new parallel kernel-independent fast multipole method. In *SC'2003 Conference CD*, Phoenix, AZ, November 2003. IEEE/ACM SIGARCH.